Introduction to XSD

XML Schema Definition

A Beginner's Guide: Writing Schemas and Validating XML

1. What is XSD?

XSD (XML Schema Definition) is the modern, powerful way to define the structure, content, and data types of an XML document. It is the recommended replacement for DTD. Unlike DTD, XSD is itself written in XML, which makes it easier to read, write, and process.

Why XSD is better than DTD

- XSD supports data types: you can say a value must be an integer, date, email, etc.
- XSD is written in XML — no new syntax to learn
- XSD supports namespaces for combining XML vocabularies
- XSD allows min/max occurrence constraints (more flexible than DTD's +, *, ?)
- XSD supports default and fixed values for elements (not just attributes)
- XSD can define reusable types and complex structures

2. DTD vs XSD — Side by Side

Here is a quick comparison to help you understand how XSD improves on DTD:

Feature	DTD	XSD
Language	Custom syntax (not XML)	Pure XML syntax
Data types	No data types — only text	Rich built-in types: integer, date, boolean…
Namespaces	Not supported	Fully supported
Reusability	Very limited	Types can be defined and reused anywhere
Occurrence	Only ?, +, *	minOccurs / maxOccurs with exact numbers

Default values	Only on attributes	On both elements and attributes
Complexity	Simple but limited	More powerful — steeper learning curve
File extension	.dtd	.xsd
Linking to XML	<!DOCTYPE ... SYSTEM>	xsi:schemaLocation or noNamespaceSchemaLocation

3. Structure of an XSD File

Every XSD file starts with a root <xs:schema> element. All schema declarations go inside this element.

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <!-- Your element and type declarations go here -->

</xs:schema>
```

Key points about xs:schema

- xmlns:xs="http://www.w3.org/2001/XMLSchema" is the mandatory namespace declaration
- The prefix xs: is conventional — you could use xsd: instead, but xs: is most common
- Everything inside the schema must use the xs: prefix
- The file is saved with a .xsd extension

4. Linking an XSD to an XML File

Unlike DTD (which uses <!DOCTYPE>), XSD is linked through attributes on the root XML element.

4.1 Without Namespace (simplest — recommended for beginners)

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="bookstore.xsd">
```

```
  <book>
    <title>Learning XML</title>
    <author>Erik Ray</author>
    <price>29.99</price>
  </book>
</bookstore>
```

Explanation

xmlns:xsi — declares the XML Schema Instance namespace (always required to use xsi: attributes)

xsi:noNamespaceSchemaLocation — points to the .xsd file (use this when your XML has no namespace)

Both attributes go on the ROOT element of your XML document

4.2 With Namespace (for advanced use)

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore
  xmlns="http://www.example.com/books"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.example.com/books bookstore.xsd">

  <!-- content -->
</bookstore>
```

Note for beginners

Namespaces can be confusing at first. Start with noNamespaceSchemaLocation.

Only use schemaLocation when your XML explicitly declares a target namespace.

5. Declaring Elements in XSD

5.1 Simple Elements (text content only)

A simple element contains only text — no child elements and no attributes.

```
<!-- Syntax -->
<xs:element name="element-name" type="xs:dataType"/>

<!-- Examples -->
```

```
<xs:element name="title"    type="xs:string"/>
<xs:element name="price"    type="xs:decimal"/>
<xs:element name="year"     type="xs:integer"/>
<xs:element name="pubDate"  type="xs:date"/>
<xs:element name="inStock"  type="xs:boolean"/>
```

5.2 Built-in XSD Data Types

XSD Type	Example Values	Description
`xs:string`	"Hello World"	Any text
`xs:integer`	42, -7, 0	Whole numbers (positive, negative, zero)
`xs:decimal`	3.14, 19.99, -0.5	Numbers with decimal places
`xs:boolean`	true, false, 1, 0	Boolean true/false values
`xs:date`	2024-12-25	Date in YYYY-MM-DD format
`xs:time`	14:30:00	Time in HH:MM:SS format
`xs:dateTime`	2024-12-25T14:30:00	Combined date and time
`xs:positiveInteger`	1, 2, 3…	Integers greater than zero
`xs:nonNegativeInteger`	0, 1, 2…	Integers >= 0
`xs:anyURI`	http://example.com	A URI/URL address

5.3 Default and Fixed Values

```
<!-- default: value used if element is absent -->
<xs:element name="edition"  type="xs:string"  default="1st"/>

<!-- fixed: element must have exactly this value if present -->
<xs:element name="currency" type="xs:string"  fixed="USD"/>
```

6. Declaring Attributes in XSD

Attributes are declared using xs:attribute inside a complex type. They always go at the end of the type definition.

```
<!-- Syntax -->
```

```
<xs:attribute name="attr-name" type="xs:dataType" use="required|optional"
default="value"/>

<!-- Examples -->
<xs:attribute name="id"       type="xs:string"   use="required"/>
<xs:attribute name="lang"     type="xs:string"   use="optional"
default="en"/>
<xs:attribute name="version"  type="xs:decimal"  use="required"/>
<xs:attribute name="category" type="xs:string"   use="optional"/>
```

use value	Meaning
`use="required"`	The attribute is mandatory — XML is invalid without it
`use="optional"`	The attribute may or may not be present (default behavior)
`use="prohibited"`	The attribute must NOT appear in the XML document

7. Complex Types — Elements with Children or Attributes

A complex type is used when an element contains child elements, attributes, or both. This is the most important concept in XSD.

7.1 Sequence — children in a fixed order

xs:sequence means child elements must appear in exactly the order listed (same as , in DTD).

```
<xs:element name="book">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="title"   type="xs:string"/>
      <xs:element name="author"  type="xs:string"/>
      <xs:element name="price"   type="xs:decimal"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

7.2 All — children in any order

xs:all means child elements can appear in any order, but each may appear at most once.

```
<xs:element name="person">
  <xs:complexType>
```

```
    <xs:all>
      <xs:element name="firstname"  type="xs:string"/>
      <xs:element name="lastname"   type="xs:string"/>
      <xs:element name="email"      type="xs:string"/>
    </xs:all>
  </xs:complexType>
</xs:element>
```

7.3 Choice — exactly one child

xs:choice means exactly one of the listed child elements must appear (same as | in DTD).

```
<xs:element name="contact">
  <xs:complexType>
    <xs:choice>
      <xs:element name="phone"  type="xs:string"/>
      <xs:element name="email"  type="xs:string"/>
      <xs:element name="fax"    type="xs:string"/>
    </xs:choice>
  </xs:complexType>
</xs:element>
```

7.4 Complex Type with Attributes

To add attributes to a complex type, add xs:attribute declarations after the content model.

```
<xs:element name="book">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="title"   type="xs:string"/>
      <xs:element name="author"  type="xs:string"/>
      <xs:element name="price"   type="xs:decimal"/>
    </xs:sequence>
    <!-- Attributes go AFTER the content model -->
    <xs:attribute name="isbn"     type="xs:string"  use="required"/>
    <xs:attribute name="edition"  type="xs:integer" use="optional"
default="1"/>
  </xs:complexType>
</xs:element>
```

8. Controlling Occurrences: minOccurs and maxOccurs

XSD gives you precise control over how many times a child element can appear. These are specified on individual xs:element declarations inside a compositor.

Attribute	Meaning
`minOccurs="0"`	Element is optional (can appear 0 times)
`minOccurs="1"`	Element must appear at least once (default)
`minOccurs="N"`	Element must appear at least N times
`maxOccurs="1"`	Element can appear at most once (default)
`maxOccurs="N"`	Element can appear at most N times
`maxOccurs="unbounded"`	Element can appear any number of times (no upper limit)

DTD equivalents

(element) = minOccurs="1" maxOccurs="1" (default — exactly once)
(element?) = minOccurs="0" maxOccurs="1" (optional)
(element+) = minOccurs="1" maxOccurs="unbounded"
(element*) = minOccurs="0" maxOccurs="unbounded"

```
<ts:element name="bookstore">
  <ts:complexType>
    <ts:sequence>
      <!-- At least 1 book, no upper limit -->
      <ts:element name="book" minOccurs="1" maxOccurs="unbounded">
        <ts:complexType>
          <ts:sequence>
            <ts:element name="title"    type="ts:string"/>
            <!-- author is optional -->
            <ts:element name="author"   type="ts:string" minOccurs="0"/>
            <!-- between 1 and 3 reviews allowed -->
            <ts:element name="review"   type="ts:string" minOccurs="1"
maxOccurs="3"/>
          </ts:sequence>
        </ts:complexType>
      </ts:element>
    </ts:sequence>
  </ts:complexType>
</ts:element>
```

9. Restrictions — Constraining Values

XSD allows you to restrict the allowed values for an element using xs:restriction. This is one of the most powerful features of XSD.

9.1 Restricting String Length

```
<xs:element name="username">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:minLength value="3"/>
      <xs:maxLength value="20"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

9.2 Restricting Numeric Range

```
<xs:element name="age">
  <xs:simpleType>
    <xs:restriction base="xs:integer">
      <xs:minInclusive value="0"/>
      <xs:maxInclusive value="120"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

9.3 Enumeration — Allowed Values List

```
<xs:element name="grade">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:enumeration value="A"/>
      <xs:enumeration value="B"/>
      <xs:enumeration value="C"/>
      <xs:enumeration value="D"/>
      <xs:enumeration value="F"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

9.4 Pattern — Regular Expression

```
<!-- Phone must match format: 3 digits - 4 digits, e.g. 555-1234 -->
<xs:element name="phone">
  <xs:simpleType>
    <xs:restriction base="xs:string">
```

```
      <xs:pattern value="[0-9]{3}-[0-9]{4}"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>

<!-- Postal code: 5 digits -->
<xs:element name="zip">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="[0-9]{5}"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

9.5 Restriction Facets Summary

Facet	Applies to	Description
`xs:minLength`	strings	Minimum number of characters
`xs:maxLength`	strings	Maximum number of characters
`xs:length`	strings	Exact number of characters required
`xs:pattern`	strings	Regular expression the value must match
`xs:enumeration`	any type	One of a fixed set of allowed values
`xs:minInclusive`	numbers/dates	Minimum value (value >= this)
`xs:maxInclusive`	numbers/dates	Maximum value (value <= this)
`xs:minExclusive`	numbers/dates	Value must be strictly greater than this
`xs:maxExclusive`	numbers/dates	Value must be strictly less than this
`xs:totalDigits`	decimals	Maximum total number of digits
`xs:fractionDigits`	decimals	Maximum number of decimal places
`xs:whiteSpace`	strings	How whitespace is handled (preserve/replace/collapse)

10. Named Types — Reusable Definitions

In XSD, you can define a type once and reuse it multiple times. This keeps your schema clean and avoids repetition.

10.1 Named Simple Types

```
<!-- Define the type once at the top level -->
<xs:simpleType name="gradeType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="A"/>
    <xs:enumeration value="B"/>
    <xs:enumeration value="C"/>
    <xs:enumeration value="D"/>
    <xs:enumeration value="F"/>
  </xs:restriction>
</xs:simpleType>

<!-- Reuse it in multiple elements -->
<xs:element name="midterm"  type="gradeType"/>
<xs:element name="final"    type="gradeType"/>
<xs:element name="quiz"     type="gradeType"/>
```

10.2 Named Complex Types

```
<!-- Define a reusable complex type -->
<xs:complexType name="addressType">
  <xs:sequence>
    <xs:element name="street"   type="xs:string"/>
    <xs:element name="city"     type="xs:string"/>
    <xs:element name="country"  type="xs:string"/>
  </xs:sequence>
</xs:complexType>

<!-- Use it in different elements -->
<xs:element name="homeAddress"   type="addressType"/>
<xs:element name="workAddress"   type="addressType"/>
<xs:element name="shippingAddress" type="addressType"/>
```

11. Complete Example — Student Grade Report

Let's recreate the student gradebook from the DTD guide using XSD, with added data type constraints.

Step 1 — Write the XSD file (grades.xsd)

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <!-- Reusable grade type: only A, B, C, D, or F allowed -->
  <xs:simpleType name="gradeType">
```

```
    <xs:restriction base="xs:string">
      <xs:enumeration value="A"/>
      <xs:enumeration value="B"/>
      <xs:enumeration value="C"/>
      <xs:enumeration value="D"/>
      <xs:enumeration value="F"/>
    </xs:restriction>
  </xs:simpleType>

  <!-- Root element -->
  <xs:element name="gradebook">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="student" minOccurs="1" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="name"   type="xs:string"/>
              <xs:element name="course" minOccurs="1" maxOccurs="unbounded">
                <xs:complexType>
                  <xs:sequence>
                    <xs:element name="grade" type="gradeType"/>
                  </xs:sequence>
                  <xs:attribute name="code" type="xs:string" use="required"/>
                </xs:complexType>
              </xs:element>
            </xs:sequence>
            <xs:attribute name="id" type="xs:ID" use="required"/>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

Step 2 — Write the XML file (grades.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
<gradebook
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="grades.xsd">

  <student id="S001">
    <name>Alice Johnson</name>
    <course code="CS101"><grade>A</grade></course>
    <course code="MATH201"><grade>B</grade></course>
  </student>

  <student id="S002">
    <name>Bob Smith</name>
```

```
    <course code="CS101"><grade>B</grade></course>
    <course code="ENG101"><grade>A</grade></course>
  </student>

</gradebook>
```

What the XSD enforces automatically

- Each student must have an id (xs:ID — must be unique across the document)
- Students must have at least one course (minOccurs="1")
- Each course must have a code attribute
- Grade values are restricted to: A, B, C, D, or F (enumeration)
- A grade of 'X' or 'Pass' would fail validation

12. Validating Your XML Against XSD

To check if your XML is valid according to your XSD, you can use several free tools:

Tool / Method	How to use
`Online: freeformatter.com`	Paste your XML and XSD — click Validate
`Online: xmlvalidation.com`	Upload or paste both files — get instant feedback
`VS Code + XML extension`	Install 'XML' by Red Hat — validates automatically on save
`Notepad++ + XML Tools`	Plugin > XML Tools > Validate Now
`oXygen XML Editor`	Professional IDE — best for serious XML/XSD work
`Java: javax.xml.validation`	Built-in Java library — for programmatic validation

13. Quick Reference Card

XSD Declaration	Purpose
`<xs:schema xmlns:xs="...">`	Root element — every XSD file starts with this
`<xs:element name="n" type="xs:string"/>`	Simple element — text only
`<xs:element name="n" minOccurs="0"/>`	Optional element

`<xs:element name="n" maxOccurs="unbounded">`	Repeating element (no upper limit)
`<xs:complexType>`	Declare an element with children or attributes
`<xs:sequence>`	Children must appear in listed order
`<xs:all>`	Children can appear in any order
`<xs:choice>`	Exactly one of the listed children must appear
`<xs:attribute name="n" use="required"/>`	Required attribute on an element
`<xs:simpleType><xs:restriction>`	Restrict the allowed values of an element
`<xs:enumeration value="X"/>`	Add an allowed value to an enumeration
`<xs:minInclusive value="0"/>`	Set minimum numeric value
`<xs:maxInclusive value="100"/>`	Set maximum numeric value
`<xs:pattern value="[A-Z]{3}"/>`	Restrict value by regular expression

Summary — What You Learned

1. XSD is written in XML and is the modern, recommended way to validate XML documents
2. XSD files start with <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
3. Link XSD to XML using xsi:noNamespaceSchemaLocation on the root element
4. Use xs:element with xs:complexType for elements that contain children or attributes
5. Use xs:sequence, xs:all, xs:choice to control child element structure
6. Control repetition precisely with minOccurs and maxOccurs
7. Restrict values using xs:restriction with facets like enumeration, pattern, minInclusive
8. Define named types to reuse across the schema and keep your XSD clean